

Seminar 4

Last week a **type** was built from constructors. This week a **claim** is:

`Buyable 13` holds because there is a way to derive it, and `Buyable 7` fails because there is none.

Two moves, and the homework is made of them: **build** a derivation, and **take one apart**. Round 1 is $\leq$, straight from the homework. Rounds 2–5 are a rule set you have not seen — 3¢ and 5¢ stamps.

Round 1 – `≤` is three lines of rules

```
#check @Nat.le.refl    -- ∀ {n}, n.le n
#check @Nat.le.step    -- ∀ {n m}, n.le m → n.le m.succ

theorem two_le_five_term : 2 ≤ 5 :=
  Nat.le.step (Nat.le.step (Nat.le.step Nat.le.refl))

theorem two_le_five : 2 ≤ 5 := by
  repeat constructor
```

the same derivation, twice

```
-- 2 ≤ 2    refl
-- 2 ≤ 3    step
-- 2 ≤ 4    step
-- 2 ≤ 5    step
```

`repeat constructor` found that term by trying `refl`, then `step`, at every goal. It is proof search, and slide 8 is where it stops working.

zero_le' – nothing to take apart

```
theorem zero_le' (n : Nat) : 0 ≤ n := by
  induction n with
  | zero => exact Nat.le.refl
  | succ n ih => exact Nat.le.step ih
```

the **SUCC** goal

```
n : ℕ
ih : 0 ≤ n
⊢ 0 ≤ n + 1
```

Induction on the number, because $0 \leq n$ is what we must build, not something we were handed. When the derivation is a hypothesis instead, induct on *it* — round 4.

le_of_succ_le_succ — homework ex03

```
theorem le_of_succ_le_succ (n m : Nat) :  
  n + 1 ≤ m + 1 → n ≤ m := by  
  induction m with  
  | zero =>  
    intro h  
    cases h with  
    | refl => exact Nat.le.refl  
    | step h' => cases h'  
  | succ m ih =>  
    intro h  
    cases h with  
    | refl => exact Nat.le.refl  
    | step h' => exact Nat.le.step (ih h')
```

case zero.step	case succ.step
$h' : (n + 1).le\ 0$	$h' : (n + 1).le\ (m + 1)$
$\vdash n \leq 0$ -- no rule for this	$\vdash n \leq m + 1$ -- ih $h' : n \leq m$

One of the four cases vanishes: **cases** on an impossible hypothesis leaves zero goals.

le_myMax – homework ex07, the if

```
def myMax (n m : Nat) : Nat :=  
  if n ≤ m then m else n
```

```
theorem le_myMax (n m : Nat) :  
  n ≤ myMax n m ∧ m ≤ myMax n m := by  
  unfold myMax  
  split  
  · exact ⟨n ≤ m, Nat.le_refl m⟩  
  · exact ⟨Nat.le_refl n, Nat.le_of_not_le <¬ n ≤ m>⟩
```

after `split`

```
-- goal 1, with h+ : n ≤ m  
⊢ n ≤ m ∧ m ≤ m  
-- goal 2, with h+ : ¬n ≤ m  
⊢ n ≤ n ∧ m ≤ n
```

`split` is `cases` for an `if`: each branch keeps the condition that selected it, inaccessibly named — `<n ≤ m>` asks for it by type. `by_cases h : n ≤ m <=> simp [h]` does the same thing with a name you chose.

Round 2 — a rule set you have not seen

```
inductive Buyable : Nat → Prop where
| zero : Buyable 0
| add3 {n : Nat} : Buyable n → Buyable (n + 3)
| add5 {n : Nat} : Buyable n → Buyable (n + 5)
```

What postage you can make from 3¢ and 5¢ stamps. Three rules, the same shape as `MyEven` — one base, two steps.

Build `Buyable 3`, `Buyable 9`, then `Buyable 13`.

The easy two

```
theorem buyable_three : Buyable 3 := .add3 .zero
```

```
theorem buyable_nine : Buyable 9 :=  
  .add3 (.add3 (.add3 .zero))
```

```
-- the lecture's search, unchanged
```

```
example : Buyable 9 := by
```

```
  repeat first | exact Buyable.zero
```

```
    | apply Buyable.add3 | apply Buyable.add5
```

what `apply` had to solve

```
⊢ Buyable 9  
-- apply Buyable.add3 needs  
--   ?n + 3 =?= 9  
-- unification: ?n := 6
```

A numeral is not a constructor pattern, and it does not have to be: Lean solves `?n + 3 =?= 9` by arithmetic. This is the same move that let `cases` work on `MyEven 3`.

The same search on 13

```
example : Buyable 13 := by
  repeat first | exact Buyable.zero | apply Buyable.add3 | apply Buyable.add5
```

```
error: unsolved goals
├ Buyable 1
```

```
13 --add3--> 10 --add3--> 7 --add3--> 4 --add3--> 1   stuck
13 --add5-->  8 --add5--> 3 --add3--> 0   ✓ (never tried)
```

repeat commits: it takes the first rule that applies and never comes back. `add3` applies to 13, so the search walks straight into 1 and gives up there. `aesop` fails on this goal too.

buyable_thirteen, two ways

```
theorem buyable_thirteen : Buyable 13 :=  
  .add5 (.add5 (.add3 .zero))
```

```
example : Buyable 13 := by  
  solve_by_elim [Buyable.zero, Buyable.add3, Buyable.add5]
```

the term, read outwards

```
.zero      -- 0  
.add3 ...  -- 3  
.add5 ...  -- 8  
.add5 ...  -- 13
```

Either you know the answer ($13 = 3 + 5 + 5$, so write it), or you ask for a search that backtracks. `solve_by_elim` retries the other rules; `repeat` never does.

Round 3 — take a derivation apart

$\neg \text{Buyable } 1$, $\neg \text{Buyable } 4$, $\neg \text{Buyable } 7$.

There is nothing to compute here yet and no formula to appeal to — only the question **which rule could have concluded this**. `cases` asks it, and keeps only the answers.

Draw the tree for 7 on paper first: two rules can conclude it.

One level, then two

```
theorem not_buyable_one : ¬ Buyable 1 := by
  intro h
  cases h
```

```
theorem not_buyable_four : ¬ Buyable 4 := by
  intro h
  cases h with
  | add3 h1 => cases h1
```

why `cases h` alone finishes 1

```
-- zero:  1 == 0      x
-- add3:   1 == ?n + 3  x
-- add5:   1 == ?n + 5  x
-- no rule survives ⇒ no goals
```

For 4 exactly one rule survives (`4 = 1 + 3`), and it hands back `Buyable 1` — the previous theorem's statement, one `cases` away from done.

¬ Buyable 7 — the tree, and the real goals

```
theorem not_buyable_seven : ¬ Buyable 7 := by
  intro h
  cases h with
  | add3 h4 =>
    cases h4 with
    | add3 h1 => cases h1
    | add5 h2 => cases h2
```

```
case add3
h4 : Buyable (0 + 4)
⊢ False
```

```
case add3.add3
h1 : Buyable (0 + 1)
⊢ False
```

```
case add5
h2 : Buyable (0 + 2)
⊢ False
```

Two branches at the root ($7 = 4 + 3$ and $7 = 2 + 5$), one below `add3`, none below the rest. Four `cases` and the whole search space is exhausted — that is what "7 is not buyable" means. Slide 22 gets the same fact for free.

Round 4 — induct over a derivation

```
buyable_add      : Buyable n → Buyable m → Buyable (n + m)
buyable_three_mul : Buyable (3 * a)
buyable_five_mul  : Buyable (5 * b)

buyable_iff_exists : Buyable n ↔ ∃ a b, n = 3 * a + 5 * b
```

`cases` gave one goal per rule. `induction` gives the same goals **plus an** `ih` wherever a rule had a premise. The rules are the cases; that is all there is to it.

buyable_add – induct on hm, not on m

```
theorem buyable_add (n m : Nat)
  (hn : Buyable n) (hm : Buyable m) : Buyable (n + m) := by
  induction hm with
  | zero => simp using hn
  | add3 _ ih =>
    rw [← Nat.add_assoc]
    exact Buyable.add3 ih
  | add5 _ ih =>
    rw [← Nat.add_assoc]
    exact Buyable.add5 ih
```

the `add3` case

```
hn : Buyable n
ih : Buyable (n + n+)
⊢ Buyable (n + (n+ + 3))
-- rw [← Nat.add_assoc]
⊢ Buyable (n + n+ + 3)
```

The goal is one `add3` away from `ih` — but only once the brackets match. This is the lecture's `even_add` with two step rules instead of one.

buyable_three_mul – back to induction on a number

```
theorem buyable_three_mul (a : Nat) : Buyable (3 * a) := by
  induction a with
  | zero => exact Buyable.zero
  | succ a ih =>
    rw [Nat.mul_succ]
    exact Buyable.add3 ih
```

the **SUCC** goal

```
ih : Buyable (3 * a)
⊢ Buyable (3 * (a + 1))
-- rw [Nat.mul_succ]
⊢ Buyable (3 * a + 3)
```

No derivation was given, so there is none to induct on — **a** it is. Same for **buyable_five_mul**, with **add5**. The lecture's **even_two_mul**, one rule over.

buyable_iff_exists – rules against a formula

```
constructor
· intro h
  induction h with
  | zero => exact ⟨0, 0, rfl⟩
  | add3 _ ih =>
    obtain ⟨a, b, hab⟩ := ih; exact ⟨a + 1, b, by omega⟩
  | add5 _ ih =>
    obtain ⟨a, b, hab⟩ := ih; exact ⟨a, b + 1, by omega⟩
· rintro ⟨a, b, rfl⟩
  exact buyable_add _ _ (buyable_three_mul a) (buyable_five_mul b)
```

Left to right: every rule that built the derivation adds one stamp to the count, so the induction writes the formula down as it goes. Right to left: no induction at all — three lemmas already did it, and $3 * a + 5 * b$ is literally `buyable_add` applied to two of them.

Round 5 — when no rule steps by one

Every amount from 8¢ up is buyable. Try it directly:

```
theorem buyable_add_eight (n : Nat) : Buyable (n + 8) := by
  induction n with
  | zero => exact .add5 (.add3 .zero)
  | succ n ih => ?_
-- ih : Buyable (n + 8)
-- ⊢ Buyable (n + 1 + 8)
```

`ih` is one cent short and **no rule adds one cent**. The statement is too weak to carry itself forward.

Carry three at once

```
theorem buyable_eight_nine_ten (n : Nat) :  
  Buyable (n + 8) ∧ Buyable (n + 9) ∧ Buyable (n + 10) := by  
  induction n with  
  | zero => exact ⟨.add5 (.add3 .zero),  
                  .add3 (.add3 (.add3 .zero)),  
                  .add5 (.add5 .zero)⟩  
  | succ n ih =>  
    obtain ⟨h8, h9, h10⟩ := ih  
    exact ⟨h9, h10, Buyable.add3 h8⟩
```

the **SUCC** goal

```
h8 : Buyable (n + 8)    -- ... h9, h10  
⊢ Buyable (n + 1 + 8)  
  ∧ Buyable (n + 1 + 9)  
  ∧ Buyable (n + 1 + 10)
```

Two thirds of the step is shifting what you already have; the new third is **add3** on **h8**, since **$n + 11 = (n + 8) + 3$** . A stronger statement is easier to prove — it hands the induction more to work with.

From $8 \leq n$ to a derivation

```
theorem buyable_of_le (n : Nat) (h : 8 ≤ n) : Buyable n := by
  obtain ⟨k, rfl⟩ := Nat.exists_eq_add_of_le h
  rw [Nat.add_comm]
  exact (buyable_eight_nine_ten k).1
```

after `obtain`

```
k : ℕ
⊢ Buyable (8 + k)
-- rw [Nat.add_comm]
⊢ Buyable (k + 8)
```

`Nat.exists_eq_add_of_le` is the lecture's `le_iff_exists_add_eq` under its library name: a $\leq$ derivation turned into an equation, so `rfl` can substitute it away.

buyable_iff — the complete answer

```
theorem buyable_iff (n : Nat) :  
  Buyable n ↔ n = 0 ∨ n = 3 ∨ n = 5 ∨ n = 6 ∨ 8 ≤ n := by  
  constructor  
  · intro h  
    obtain {a, b, rfl} := (buyable_iff_exists n).mp h  
    rcases a with _ | _ | _ | a <;> rcases b with _ | _ | b <;> omega  
  · rintro (rfl | rfl | rfl | rfl | h)  
    · exact .zero  
    · exact .add3 .zero  
    · exact .add5 .zero  
    · exact .add3 (.add3 .zero)  
    · exact buyable_of_le n h
```

Left to right is the formula doing the work; right to left is five derivations, four of them written out by hand and the fifth from the previous slide.

Where `omega` stops

```
-- with a bare `omega` in place of the rcases line:  
error: omega could not prove the goal:  
a possible counterexample may satisfy the constraints  
  d ≥ 0  
  c ≥ 0  
  1 ≤ 3*c + 5*d ≤ 2  
where  
  c := ↑a  
  d := ↑b
```

`omega` decides linear arithmetic; "which numbers have the form $3a + 5b$ " is not that — it cannot see that 1¢ and 2¢ are out of reach. So split `a` into 0, 1, 2, ≥ 3 and `b` into 0, 1, ≥ 2 by hand: `rcases a with _ | _ | _ | a <;> rcases b with _ | _ | b <;> omega`. Every one of the twelve cases left is linear.

Decidable – the rules, compiled away

```
instance (n : Nat) : Decidable (Buyable n) :=  
  decidable_of_iff _ (buyable_iff n).symm
```

```
example : Buyable 100 := by decide  
example : ¬ Buyable 7 := by decide  
#eval decide (Buyable 7) -- false
```

what the instance is

```
-- buyable_iff n : Buyable n ↔ (a decidable claim)  
-- decidable_of_iff transports the decision  
-- procedure across the ↔
```

Round 3 spent four **cases** on **¬ Buyable 7**. With the characterisation proved, **decide** settles it — and 100, and anything else — by computing. Same bridge as the lecture's **Decidable (MyEven n)**.

What to take away

- **A rule set gives you both directions.** To prove the claim, build a derivation; to use one, `cases` it. Rules that cannot conclude the goal simply are not there.
- **induction on a derivation, not on the number** — whenever the derivation is what you were handed (`buyable_add` , `buyable_iff_exists`).
- `repeat constructor` is greedy, not a search: it walks $13 \rightarrow 10 \rightarrow 7 \rightarrow 4 \rightarrow 1$ and stops. Write the term, or use something that backtracks.
- **Strengthen the statement when no rule steps by one.** `Buyable (n + 8)` alone cannot carry an induction; the triple can.
- Rules $\rightarrow$ formula $\rightarrow$ computation is the lecture's triangle again: proving `buyable_iff` is what buys `by decide` .